Arquitectura de Spark

Contents

- Componentes de Spark
- Arquitectura de una aplicación con Spark

Componentes de Spark

Apache Spark consiste en una API (bibliotecas) orientada al programador, **mucho** más intuitiva que MapReduce. Al igual que éste, abstrae todos los detalles de comunicación de red y hardware, pero además, opera de manera similar a las consultas SQL tradicionales como si los datos fuesen tablas (distribuidas). Esto la hace fácil de usar y de aprender.

Spark ofrece APIs para cuatro lenguajes:

- Java: muy tediosa de usar (sintaxis muy verbosa) pero útil para aplicaciones legacy existentes.
- Scala: es la API más utilizada para aplicaciones en producción. Scala es un lenguaje funcional que se compila sobre la JVM (la compilación genera bytecode de Java), muy compacto y cómodo de utilizar con Spark. La mayor parte del código del propio Spark es lenguaje Scala.
- Python (paquete de Python *pyspark*)*:* API casi idéntica a la de Scala, salvo peculiaridades del lenguaje. En realidad es un wrapper (capa adicional) sobre la implementación en Scala, que es la verdaderamente distribuida. No obstante, Spark también permite ejecutar código python en los nodos del cluster.
- R (paquete de R *SparkR*): es una API muy diferente al resto, más cercana a los comandos típicos de R. Últimamente, el paquete *sparklyr* (creado por la empresa RStudio) es más útil, ofrece una mejor integración y más avanzada, y su sintaxis es más intuitiva.

La Fig. 2, tomada de [1], muestra los principales componentes de Spark.

Skip to main content

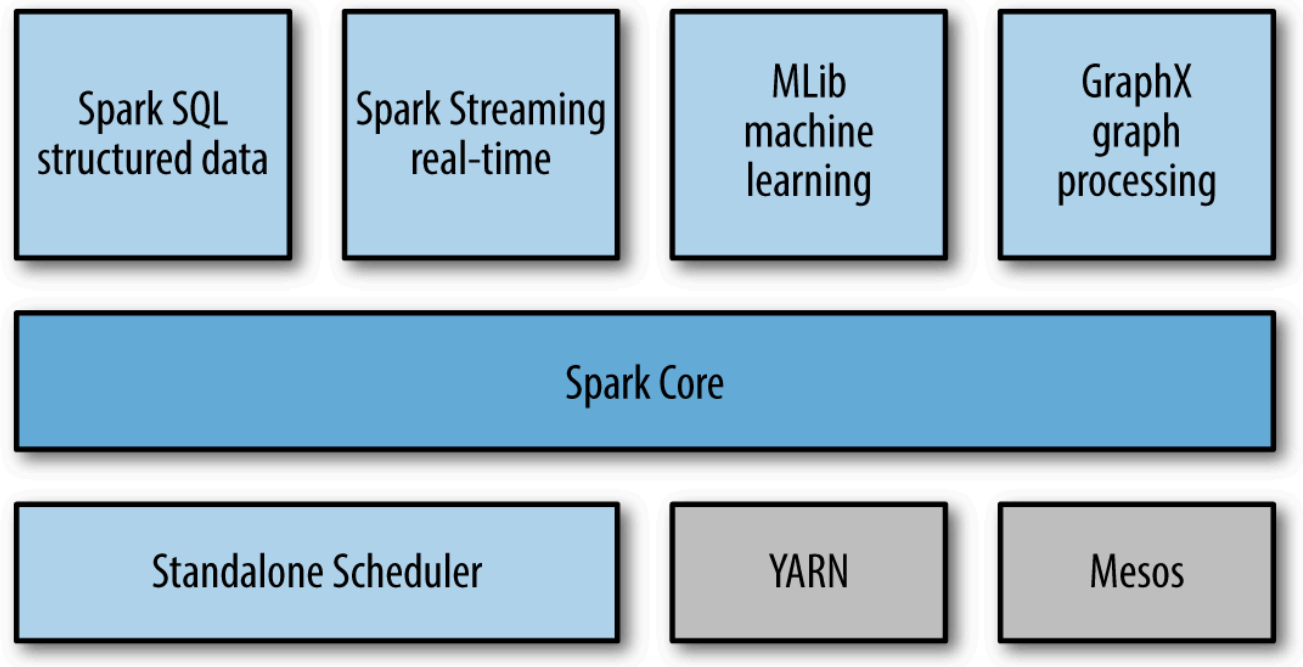


Fig 2. Módulos que componen Apache Spark

De estos, el principal es el módulo *Spark core*. En él se encuentran las abstracciones fundamentales de Spark, tales como el RDD (Resilient Distributed Dataset), del que hablaremos más adelante, y las operaciones que se puede llevar a cabo con él. En última instancia, todas las operaciones que se pueden hacer con Spark, independientemente del módulo utilizado o la API donde se encuentren, se traducen a operaciones sobre RDDs, que son las únicas que Spark es capaz de ejecutar.

Los tres módulos inferiores representan tres gestores de recursos de un cluster sobre los que puede ejecutarse Spark. Un gestor de recursos se encarga de asignar máquinas, CPUs y memoria principal a Spark, de forma que disponga de un determinado número de nodos y en cada uno existan suficientes recursos para ejecutar la aplicación que hemos implementado con Spark. YARN y Mesos no forman parte de Spark pero son compatibles con Spark. El Standalone Scheduler es el gestor de recursos más sencillo, y viene incluido con Spark. No permite un control de grano muy fino de los recursos pero es suficiente para casos de uso simples.

El resto de módulos cumplen las siguientes funciones:

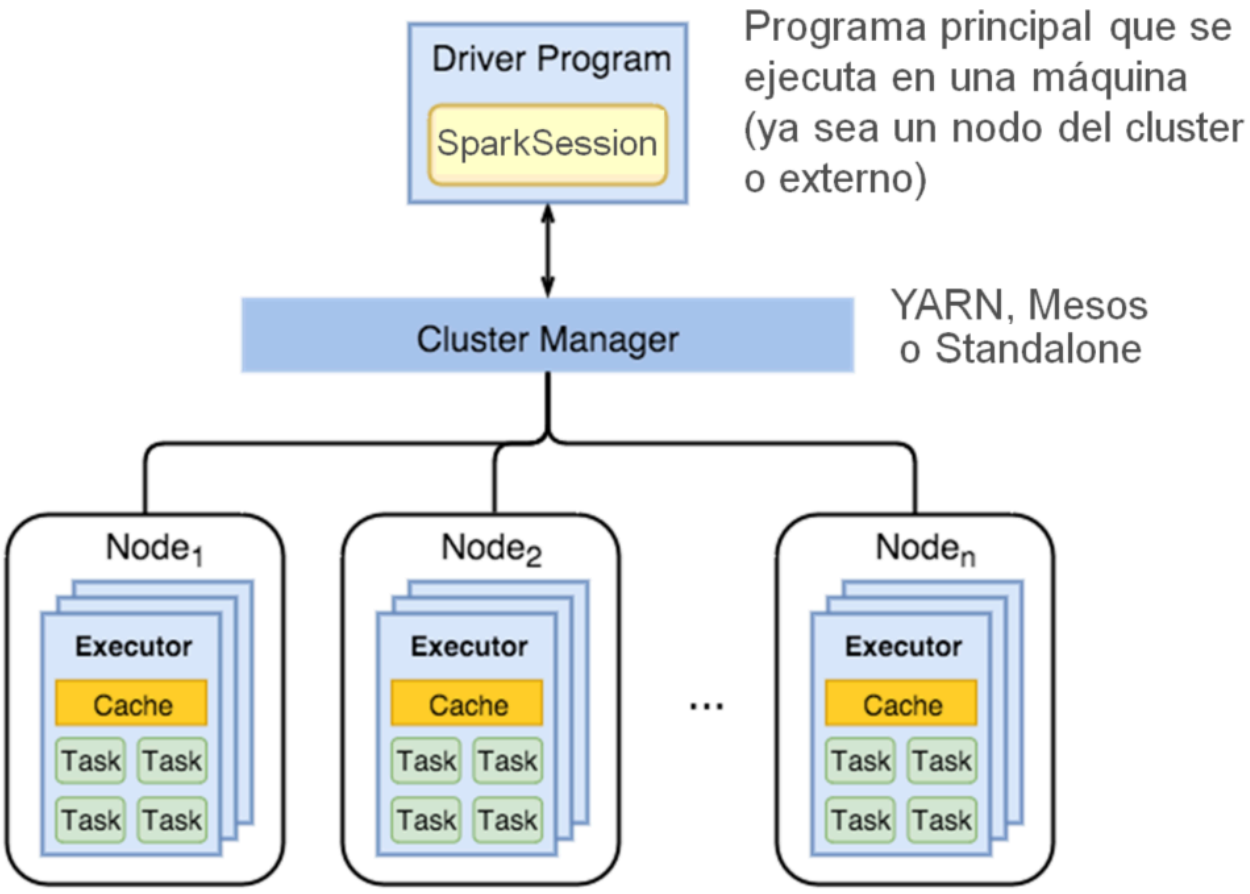
- **Spark SQL** es una API con una serie de funciones para manejar tablas de datos distribuidas, estructuradas en columnas con nombre y tipo, denominadas DataFrames. Un DataFrame proporcionan un nivel de abstracción adicional sobre un RDD, al cual recubre. Este módulo incluye también una función para ejecutar sentencias SQL sobre las mismas estructuras de datos distribuidas. Estas sentencias son analizadas y convertidas en un plan lógico que en última instancia se traduce a operaciones sobre

Skip to main content

- **Spark Streaming** es el módulo para operar de manera distribuida sobre datos en tiempo real según los vamos recibiendo (*stream* de datos). Actualmente (a partir de la versión Spark 2.0) este módulo ha sido reemplazado por *Spark Structured Streaming*, que simplifica las operaciones con flujos de datos y será el que estudiaremos en este curso.
- **Spark MLlib** contiene implementaciones distribuidas de algoritmos de Machine Learning. Aunque el módulo conserva este nombre, el paquete actual de la API se llama Spark ML y simplifica el uso de estos algoritmos puesto que actúan sobre DataFrames.
- **Spark GraphX** es el módulo de procesamiento de grafos, representados mediante RDDs. Contiene algunos algoritmos de camino mínimo y similares. Actualmente este módulo ha quedado obsoleto, y ha sido reemplazado *de facto* por el paquete GraphFrames, que representa un grafo como una pareja de DataFrames con los nodos y los arcos respectivamente. GraphFrames se desarrolló como una biblioteca externa a Spark y será la biblioteca que estudiaremos en el curso.

Arquitectura de una aplicación con Spark

La siguiente figura muestra la arquitectura de una aplicación que utiliza Spark al ejecutarse en un cluster.



Skip to main content

Fig. 3. Arquitectura de una aplicación en Spark

Cuando escribimos una aplicación en Spark, en realidad estamos escribiendo una aplicación **secuencial** (no paralela) utilizando la biblioteca de Spark para el lenguaje en el que estemos programando (Java, Scala, Python o R). Cuando ejecutamos el código de este programa, el proceso en el cual se ejecuta se denomina *Driver*, y se ejecuta sobre una máquina concreta, que puede incluso ser externa al cluster. Por ejemplo, podríamos escribir un programa en Spark y ejecutarlo en nuestro ordenador portátil, y desde aquí, el programa Driver podría conectarse a un cluster de Spark remoto, lo cual se denomina ejecución en modo *client* (cliente). También existe la opción de que el programa Driver sea enviado al cluster y se ejecute dentro de uno de los nodos del cluster, lo cual se denomina ejecución en modo cluster.

En el transcurso de la ejecución, normalmente necesitaremos crear un objeto que solemos guardar en una variable llamada *spark*, de la clase SparkSession de Spark. En el momento de crear este objeto, hay que indicar dónde (en qué dirección IP y en qué puerto) existe un cluster de Spark configurado o, más concretamente, dónde existe un gestor de cluster que nos pueda dar acceso a los recursos (nodos, memoria, CPUs) de un cluster. Ciertas aplicaciones, como por ejemplo ciertos *kernel* especiales para Jupyter Notebook, o la línea de comandos de Spark tanto en Scala (spark-shell) como en python (pyspark-shell) ya crean un objeto sparkSession por nosotros al arrancar la aplicación.

De esta manera, la sparkSession establece comunicación con el gestor de cluster. A partir de este momento, podemos utilizar dichos recursos *para las sentencias que requieran ejecución distribuida*.

Es importante recalcar esto último: la ejecución de un programa en Spark es secuencial, en una sola máquina, tal como sería cualquier otro programa en el lenguaje elegido, **excepto cuando el flujo de programa llega a ciertas funciones específicas de Spark que desencadenen ejecución distribuida**, que son la mayoría (pero no todas) las que forman la API de Spark. Muchas de estas funciones se aplican sobre el objeto sparkSession, y otras se aplican sobre estructuras de datos distribuidas que hayamos ido creando en el programa, como por ejemplo RDDs o DataFrames de Spark.

Configuración del objeto SparkSession

En el momento de crear el objeto sparkSession, hemos indicado una configuración con el número de nodos (en realidad, número de *executors*), la memoria RAM y número de cores físicos que necesitamos reservar en cada nodo. Los recursos concedidos por el gestor de

Skip to main content

políticas y del número de recursos que ya estuvieran ocupados en ese momento). En cualquier caso, la solicitud los deja reservados para que, llegado el momento en el que se vaya a ejecutar una operación distribuida de nuestro programa, puedan ser utilizados.

Estos recursos en un nodo constituyen lo que se denomina un *Executor*: un proceso de la JVM (Máquina Virtual de Java) que se ejecuta en el nodo y que ocupa los recursos indicados (cores, RAM, disco duro). El proceso ejecutor es creado por el gestor de cluster cuando arranca nuestra aplicación de Spark, y muere cuando la aplicación finaliza (ya sea con éxito o por alguna condición imprevista que provoca que toda la aplicación termine abruptamente). Cada ejecutor queda preparado para ejecutar *tareas* de Spark, que es la unidad mínima de ejecución de trabajos. Cada tarea require un core libre para ejecutarse, por lo que si un Executor tiene reservados 4 cores, podrá ejecutar 4 tareas en paralelo. Detallaremos esto más adelante. Cada uno de los nodos del cluster en los que se crean ejecutores se denomina *Worker*.

Para crear la sesión de Spark con estas indicaciones, se utiliza código similar a:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder\
                    .config("spark.executor.instances", "3")\
                    .config("spark.executor.cores", "4")\
                    .getOrCreate()
```

Este código creará 3 executors con 4 cores cada uno, siempre que existan recursos suficientes para hacerlo.

Algunas notas importantes:

- Podrían crearse varios executors en un mismo worker, si existen suficientes recursos en esa máquina worker, y si el gestor de recursos del cluster lo permite.
- Varias aplicaciones podrían compartir un mismo worker si existen suficientes recursos en esa máquina worker. Cada aplicación creará sus propios executors.
- En versiones sucesivas de Spark se introdujo la característica llamada ***dynamic allocation*** que permite crear y destruir **executors** en tiempo de ejecución en función de las necesidades de cómputo (por ejemplo si Spark detecta pocos datos al crear en DAG de un procesamiento o detecta un procesamiento muy pesado computacionalmente)
- Además, el proveedor Databricks tiene un servicio específico de *auto-escalado de clusters*, que unido a un proveedor cloud, permite añadir o quitar **workers** (y por tanto,

Skip to main content

- En los **notebooks** proporcionados **por proveedores cloud concretos**, ciertos kernel configurables en esos notebooks ya traen una Spark Session configurada y lista para usar.
    - Un kernel simple de Jupyter como el que podemos desplegar en nuestro portátil no ofrece esta funcionalidad, que es propia de cada proveedor
    - En el caso particular de Databricks, se crea **un executor en cada máquina del cluster al cual asociamos el notebook**, y en principio ocupa todo el worker. Más detalles en la documentación oficial: https://spark.apache.org/docs/latest/job-scheduling.html
- Habitualmente, las ejecuciones batch de flujos de procesamientos con Spark en producción, tales como procesos de ingesta, consolidación/fusión de datos de varias fuentes o preprocesado de datos no suelen realizarse en un notebook, con lo que la creación de la sesión de Spark es un primer paso imprescindible.

Cuando creamos un cluster en una única máquina, que es la misma en la que se está ejecutando el propio Driver, se suele utilizar la opción `.master("local[4]")` siendo 4 (o cualquier número) el número de cores que deseamos utilizar para el único executor que se creará. Para usar todos los disponibles, se utiliza `local[*]`.